

LAB 01 – Criptografia de Chat P2P utilizando AES-GCM

Aluno: Eduardo Caversan da Silva Rocha - 2475391

Disciplina: Segurança da Informação

Atividade: Laboratório 01 – Criptografia de Chat utilizando AES-GCM

1. Introdução

Este laboratório teve como objetivo implementar criptografia em um sistema de chat P2P em

Python utilizando o algoritmo AES-GCM disponibilizado pela biblioteca cryptography. O sistema

original foi modificado para que todas as mensagens enviadas entre os clientes sejam criptografadas antes da transmissão e descriptografadas ao serem recebidas.

2. Metodologia

Para realizar a criptografia das mensagens foi utilizado o algoritmo AES-GCM (Advanced Encryption Standard – Galois/Counter Mode). Este modo de operação fornece confidencialidade

e autenticação dos dados. Foi utilizada: Chave AES de 256 bits Nonce de 96 bits Biblioteca Python: cryptography A chave e o nonce foram definidos de forma fixa no código conforme solicitado na atividade.

3. Código do Client.py

```
import socket
import sys
import time
import threading
import select
import traceback
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

KEY =
b'\x9c\x14\xef\xd3\x7a\x8b\x52\x91\x3f\x2e\xc1\x99\x47\xac\x6d\x0f\x72\x81\xab\x53\x0e\x94\x5c\x1a\xf7\x63\x2b\x88\x4d\xda\x11\x6e'
NONCE = b'\x6a\x3f\x91\x2c\x8e\x74\x19\xb0\x5d\xfa\x44\xce'

class Server(threading.Thread):

    def initialise(self, receive):
        self.receive = receive
```

```

        self.aesgcm = AESGCM(KEY)

    def decrypt_message(self, data):
        try:
            plaintext = self.aesgcm.decrypt(NONCE, data, None)
            return plaintext.decode()
        except:
            return "[Erro ao descriptografar]"

    def run(self):
        lis = []
        lis.append(self.receive)
        while 1:
            read, write, err = select.select(lis, [], [])
            for item in read:
                try:
                    s = item.recv(1024)
                    if s != b'':
                        msg = self.decrypt_message(s)
                        print(msg + '\n>>')
                except:
                    traceback.print_exc(file=sys.stdout)
                    break

class Client(threading.Thread):

    def connect(self, host, port):
        self.sock.connect((host, port))

    def encrypt_message(self, msg):
        aesgcm = AESGCM(KEY)
        ciphertext = aesgcm.encrypt(NONCE, msg, None)
        return ciphertext

    def client(self, host, port, msg):
        encrypted = self.encrypt_message(msg)
        self.sock.send(encrypted)

    def run(self):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
        try:

```

```

        host = input("Enter the server IP\n>>")
        port = int(input("Enter the server Destination Port\n>>"))
    except EOFError:
        print("Error")
        return 1

    print("Connecting\n")
    self.connect(host, port)
    print("Connected\n")
    user_name = input("Enter the User Name to be Used\n>>")
    receive = self.sock
    time.sleep(1)
    srv = Server()
    srv.initialise(receive)
    srv.daemon = True
    print("Starting service")
    srv.start()

    while 1:
        msg = input('>>')
        if msg == 'exit':
            break
        if msg == '':
            continue
        msg = user_name + ': ' + msg
        data = msg.encode()
        self.client(host, port, data)
    return (1)

if __name__ == '__main__':
    print("Starting client")
    cli = Client()
    cli.start()

```

4. Código do Server.py

```

import socket
import sys
import traceback
import threading
import select

SOCKET_LIST = []
TO_BE_SENT = []

```

```

SENT_BY = {}

class Server(threading.Thread):

    def init(self):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.sock.bind(('', 50250))
        self.sock.listen(2)
        SOCKET_LIST.append(self.sock)
        print("Server started on port 50250")

    def run(self):
        while 1:
            read, write, err = select.select(SOCKET_LIST, [], [], 0)
            for sock in read:
                if sock == self.sock:
                    sockfd, addr = self.sock.accept()
                    print(str(addr))
                    SOCKET_LIST.append(sockfd)
                else:
                    try:
                        s = sock.recv(1024)

                        if s == b'':
                            continue
                        else:

                            TO_BE_SENT.append(s)

                            SENT_BY[s] = (str(sock.getpeername()))
                    except:
                        print(str(sock.getpeername()))

class handle_connections(threading.Thread):

    def run(self):
        while 1:
            read, write, err = select.select([], SOCKET_LIST, [], 0)
            for items in TO_BE_SENT:
                for s in write:
                    try:

```

```

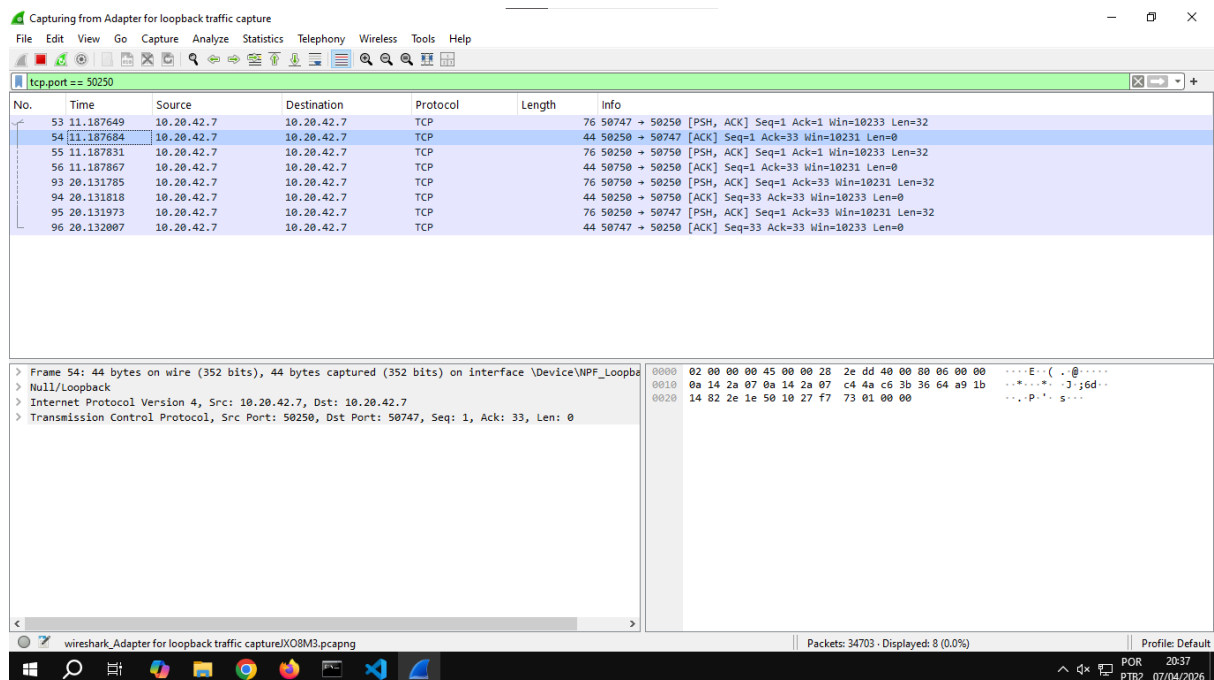
        if (str(s.getpeername()) == SENT_BY[items]):
            continue
        s.send(items)
    except:
        traceback.print_exc(file=sys.stdout)
    TO_BE_SENT.remove(items)
    del(SENT_BY[items])

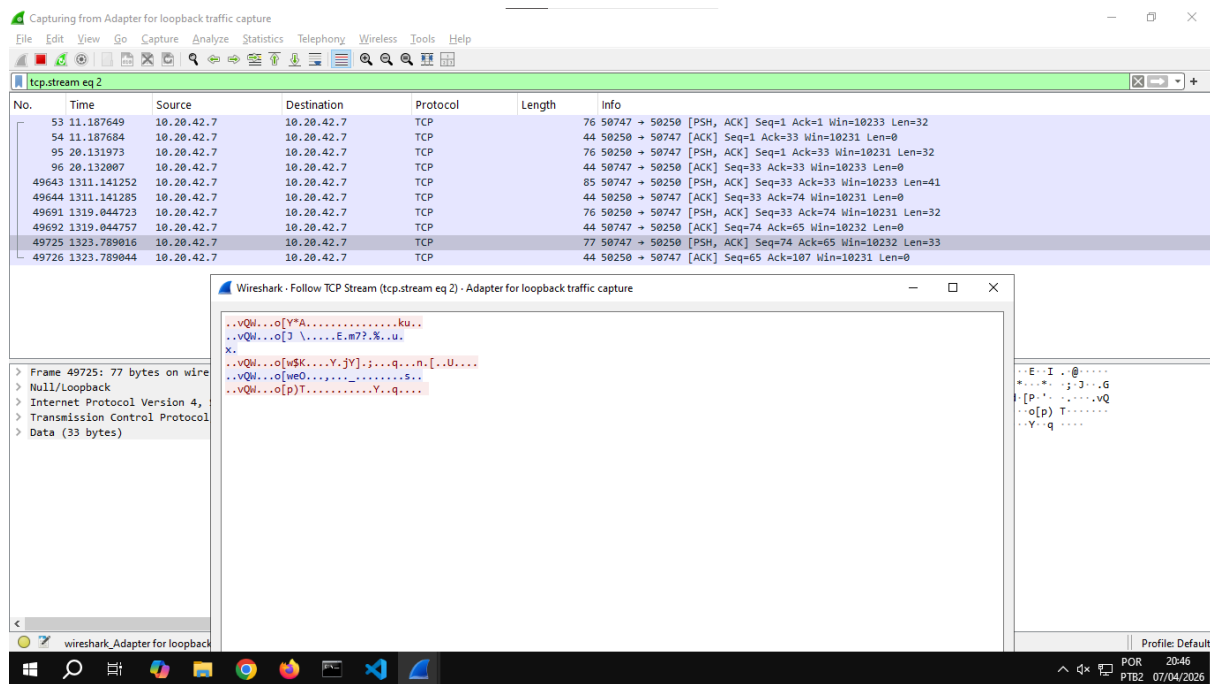
if __name__ == '__main__':
    srv = Server()
    srv.init()
    srv.start()
    handle = handle_connections()
    handle.start()

```

5. Captura de Tráfego no Wireshark

A figura abaixo apresenta a captura de tráfego realizada utilizando o software Wireshark. Observa-se que os dados transmitidos entre os clientes aparecem como bytes aparentemente aleatórios no campo Data, indicando que as mensagens foram criptografadas utilizando o algoritmo AES-GCM. A captura foi realizada filtrando os pacotes TCP da porta 50250, utilizada pelo servidor do chat.





6. Conclusão

Com a implementação da criptografia AES-GCM no cliente do chat, foi possível garantir que as mensagens transmitidas na rede não sejam enviadas em texto plano. A análise do tráfego no Wireshark confirmou que os dados capturados aparecem como bytes criptografados, impossibilitando a leitura direta das mensagens transmitidas entre os clientes.